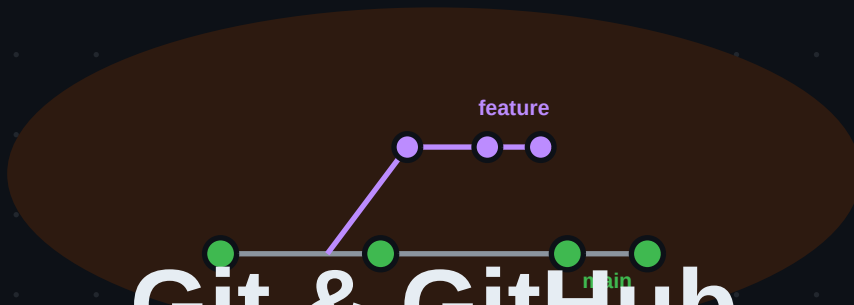


■ WHAT YOUR NOTES MISSED



Git & GitHub

Advanced Topics

Reflog · Cherry-pick · Tags · Bisect · SSH · GitHub CLI & more

Supplement to [Git_GitHub_101](#) · [Git_Github Notes](#) · [git-cheat-sheet-education](#)

Krish6115 · **GitxGithub**

. What This PDF Covers

Topics not found in your existing 3 PDFs

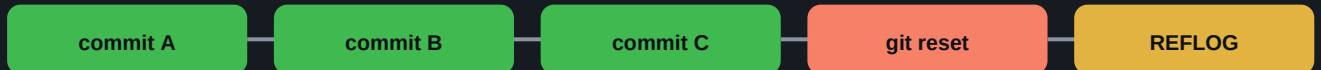
#	Topic	Why it Matters
01	git reflog	Recover ANY "lost" commit
02	git cherry-pick	Apply one commit anywhere
03	git revert	Safe undo (vs reset)
04	git commit --amend	Fix your last commit
05	git tag	Version releases (v1.0, v2.0)
06	git bisect	Binary-search a bug
07	git blame	Who wrote that line?
08	git log (advanced)	--oneline --graph --decorate
09	git clean	Nuke untracked files safely
10	git aliases	Speed up your workflow
11	SSH Keys & PAT	Auth without passwords
12	GitHub CLI (gh)	GitHub from the terminal
13	.gitignore patterns	Advanced ignore rules
14	GitHub Features	Issues, Actions, Pages, Releases

01. git reflog

The "undo everything" safety net

Every action you take in Git — commit, reset, checkout, merge — gets recorded in the reflog. Even if you accidentally delete a branch or do a hard reset, **reflog lets you go back**. It's your time machine.

Lifecycle diagram — how reflog captures every HEAD movement:



```
● ● ● # Show full history of HEAD movements
git reflog
# Output looks like:
# abc1234 HEAD@{0}: reset: moving to HEAD~1
# def5678 HEAD@{1}: commit: Fixed login bug
# ghi9012 HEAD@{2}: commit: Add user model
```

```
● ● ● # Jump back to any point in history
git checkout def5678
# or recover a lost branch:
git branch recovered-branch def5678
```

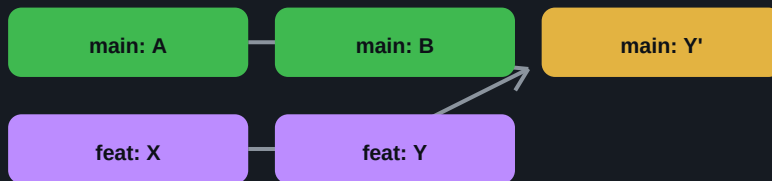
■ **git reflog only exists LOCALLY.** After 90 days Git garbage-collects old entries. Push important branches before that.

■ **After any scary 'git reset --hard' run 'git reflog' immediately — the commit is still there.**

02. git cherry-pick

Apply one specific commit to any branch

Cherry-pick lets you take **one commit from any branch** and apply it onto your current branch. Useful when a bug fix on a feature branch needs to go to main without merging the whole branch.



Y' is a copy of commit Y applied onto main.

```
● ● ● # Pick just one commit from feature-branch
git log --oneline feature-branch
# abc1234 Fix null pointer in auth
# def5678 Add new UI button

git checkout main
git cherry-pick abc1234
```

```
● ● ● # Advanced cherry-pick patterns
# Cherry-pick a range of commits:
git cherry-pick abc1234^..def5678

# Cherry-pick without auto-committing (stage only):
git cherry-pick abc1234 --no-commit
```

■ Use `--no-commit` if you want to combine multiple cherry-picked changes into one clean commit.

■ ■ Cherry-pick creates a **NEW** commit with a different hash — it's not the same commit as the original.

03. git revert

Safe undo — never rewrites history

git revert creates a new commit that undoes a previous commit. Unlike **git reset** which rewrites history, revert is **safe to use on shared/public branches** because it doesn't remove commits — it adds an inverse commit.

	git reset	git revert
What it does	Moves HEAD back, removes commits	Adds a new "undo" commit
Rewrites history?	YES	No
Safe on main/shared branch?	DANGEROUS	Safe
When to use	Local-only, before push	After pushing to remote
Undo-able?	With reflog only	Yes, revert the revert

```
● ● ● # Safely undo a commit on a shared branch
git log --oneline
# a3f1c2b Broke the login page (oops)
# 91bc4ed Added payment gateway

git revert a3f1c2b
# Opens editor → save the commit message
# Creates: Revert "Broke the login page (oops)"
```

```
● ● ● # Flags you will use
# Revert without opening editor:
git revert a3f1c2b --no-edit

# Revert a merge commit:
git revert -m 1 <merge-commit-hash>
```

■ **Never use 'git reset --hard' on commits that teammates have already pulled. Use revert instead.**

04. git commit --amend

Fix your last commit

Amend lets you **edit the last commit** — change the message, add forgotten files, or remove files. The commit gets replaced with a new one (new hash). Only safe to amend commits that **haven't been pushed yet**.

```
● ● ● # --no-edit keeps the existing message
# Fix the commit message only:
git commit --amend -m "Better commit message"
```

```
# Add a forgotten file to last commit:
git add forgotten_file.py
git commit --amend --no-edit
```

```
● ● ● # --force-with-lease is safer than --force
# Remove a file that was accidentally committed:
git reset HEAD~ forgotten.env
git commit --amend --no-edit
```

```
# Amend already-pushed commit (force required!):
git push origin branch-name --force-with-lease
```

■ **--force-with-lease is safer than --force because it fails if someone else pushed since your last pull.**

■ ■ **Never amend commits on main or shared branches. Rewrite on personal feature branches only.**

Quick Reference — When to use what:

Situation	Command
Typo in commit message	<code>git commit --amend -m 'Fixed msg'</code>
Forgot to add a file	<code>git add file && git commit --amend --no-edit</code>
Added a wrong file	<code>git reset HEAD~ file && git commit --amend --no-edit</code>

05. git tag

Marking version releases (v1.0, v2.0...)

Tags are **permanent pointers to specific commits**, typically used for version releases. Unlike branches, tags do not move. There are two types:

Type	Description	Use Case
Lightweight	Just a pointer/alias to a commit	Quick personal bookmarks
Annotated	Full object with tagger, date, message	Official releases

```
● ● ● # Creating tags
# Lightweight tag:
git tag v1.0

# Annotated tag (recommended for releases):
git tag -a v1.0 -m "First stable release"

# Tag a specific past commit:
git tag -a v0.9 abc1234 -m "Beta release"
```

```
● ● ● # Tag operations
# List all tags:
git tag

# Show tag details:
git show v1.0

# Push a single tag to remote:
git push origin v1.0

# Push ALL local tags to remote:
git push origin --tags

# Delete a local tag:
git tag -d v1.0

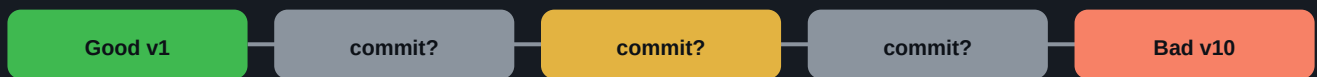
# Delete a remote tag:
git push origin --delete v1.0
```

■ **GitHub** automatically creates a **Release** when you push an annotated tag — great for open source projects.

06. git bisect

Binary-search to find which commit broke things

When a bug appears and you do not know which commit introduced it, **git bisect** automates a **binary search** through your commit history. You just mark commits as "good" or "bad" and Git narrows it down in $O(\log n)$ steps.



Git tests the middle commit each round — eliminates half the suspects every step.

```
● ● ● # Interactive bisect session
```

```
git bisect start
```

```
# Tell Git the current commit is broken:
```

```
git bisect bad
```

```
# Tell Git a known-good commit:
```

```
git bisect good v1.0
```

```
# Git checks out middle commit — test it!
```

```
# If the bug exists:
```

```
git bisect bad
```

```
# If no bug:
```

```
git bisect good
```

```
# Git keeps narrowing until it prints:
```

```
# abc1234 is the first bad commit
```

```
# When done:
```

```
git bisect reset
```

```
● ● ● # Automated bisect — no manual steps needed
```

```
# Automate with a test script (0=good, 1=bad):
```

```
git bisect start HEAD v1.0
```

```
git bisect run python test_login.py
```


07. git blame

Who wrote that line — and when?

git blame shows you, for each line of a file, the commit hash, author name, timestamp, and line content. Perfect for understanding why code exists and who to ask about it.

```
● ● ● # Understanding git blame output
git blame src/auth.js

# Output format:
# a1b2c3d4 (Krish 2024-08-15 14:32) const login = async (user) => {
# e5f6g7h8 (Alice 2024-09-01 09:14)   if (!user.email) return null;

# Blame a specific range of lines:
git blame -L 20,40 src/auth.js

# Ignore whitespace changes:
git blame -w src/auth.js

# Blame specific commit:
git blame abc1234 -- src/auth.js
```

■ On GitHub, click any file then click 'Blame' in the top-right to see blame in a nice UI.

■ ■ git blame is NOT about assigning fault — it's about understanding context and history.

08. git log (Advanced)

--oneline, --graph, --decorate and more

The basic **git log** is powerful but the flags make it *incredibly* useful for understanding project history at a glance.

Command	What it Shows
<code>git log --oneline</code>	One line per commit: hash + message
<code>git log --oneline --graph</code>	ASCII branch graph + one line
<code>git log --oneline --graph --all</code>	All branches in the graph
<code>git log --oneline --graph --decorate</code>	Includes branch/tag labels
<code>git log --author="Krish"</code>	Filter by author name
<code>git log --since="2 weeks ago"</code>	Commits in last 2 weeks
<code>git log --grep="fix"</code>	Commits with "fix" in message
<code>git log -p src/auth.js</code>	Full diff for a specific file
<code>git log --stat</code>	Files changed + line stats per commit
<code>git shortlog -sn</code>	Summary: commit count per author
<code>git log --format="%h %an %s"</code>	Custom format: hash, author, subject

```
●●● # Power user combo
# The most useful combo – pretty git graph:
git log --oneline --graph --decorate --all

# Create a permanent alias for it (see Section 10):
git config --global alias.lg "log --oneline --graph --decorate --all"

# Then just use:
git lg
```

09. git clean

Remove untracked files and directories

git clean removes files that are not tracked by Git. Useful for cleaning up build artifacts, generated files, or a messy working directory. **Unlike git restore, it cannot be undone** — these files are deleted permanently.

Flag	Effect
-n (dry run)	Preview what WOULD be deleted — always run this first!
-f (force)	Actually delete the untracked files
-d	Also remove untracked directories
-x	Also remove files ignored by .gitignore
-i	Interactive mode — confirm each file

```
● ● ● # Standard clean workflow
# Step 1: Preview (ALWAYS do this first):
git clean -n
# Would remove: build/output.jar
# Would remove: __pycache__/module.pyc

# Step 2: Clean files only:
git clean -f

# Step 3: Clean files AND directories:
git clean -fd

# Nuclear option — clean everything including .gitignored:
git clean -fdx
```

■ **git clean -fdx will delete your .env files, node_modules, build outputs — everything not tracked. No undo.**

10. git aliases

Speed up your daily Git workflow

Git aliases are shortcuts you define for longer Git commands. Once set globally, they work in every repo on your machine forever.

```
● ● ● # Set these once, use forever
```

```
# Useful aliases to add right now:
```

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "restore --staged"
git config --global alias.undo "reset HEAD~1 --mixed"
```

```
● ● ● # Aliases in action
```

```
# Usage after setting aliases:
```

```
git st          # instead of git status
git co main     # instead of git checkout main
git br          # instead of git branch
git lg          # beautiful log graph
git last        # see your last commit
git unstage file # instead of git restore --staged file
git undo        # soft-undo last commit
```

```
● ● ● # Managing aliases
```

```
# View all your current aliases:
```

```
git config --global --list | grep alias
```

```
# Or edit ~/.gitconfig directly:
```

```
cat ~/.gitconfig
```

11. SSH Keys & Personal Access Tokens

Auth without passwords

GitHub removed password authentication for Git operations in August 2021. You now need either SSH keys or a Personal Access Token (PAT). SSH is generally preferred for daily use.

SSH Key Setup:

```
● ● ● # One-time SSH setup
# Step 1: Generate SSH key (use your GitHub email):
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept default location (~/.ssh/id_ed25519)
# Set a passphrase (recommended)

# Step 2: Add key to SSH agent:
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Step 3: Copy your public key:
cat ~/.ssh/id_ed25519.pub
# Copy the output
```

■ ■ Then go to GitHub → Settings → SSH and GPG keys → New SSH key → Paste the public key.

```
● ● ● # Using SSH
# Step 4: Test the connection:
ssh -T git@github.com
# Success: "Hi Krish6115! You have successfully authenticated."

# Step 5: Clone using SSH (not HTTPS):
git clone git@github.com:Krish6115/LeetCode.git

# Switch existing HTTPS remote to SSH:
git remote set-url origin git@github.com:Krish6115/LeetCode.git
```

Personal Access Token (PAT) — When to use:

PAT is best for **scripting, CI/CD, or one-off operations**. Go to GitHub → Settings → Developer settings → Personal access tokens → Generate new token. Set scopes (repo, workflow) and copy the token. Use it as your password in HTTPS clones.

12. GitHub CLI (gh)

Full GitHub workflow from the terminal

GitHub CLI (gh) lets you manage pull requests, issues, releases, and repos entirely from the command line — no browser needed. Install from cli.github.com.

Category	Command	What it Does
Auth	gh auth login	Authenticate with GitHub
Repo	gh repo clone Krish6115/LC	Clone a repo
Repo	gh repo create my-project	Create a new GitHub repo
PR	gh pr create	Create a pull request interactively
PR	gh pr list	List open PRs in current repo
PR	gh pr checkout 42	Check out PR #42 locally
PR	gh pr merge 42	Merge PR #42
Issue	gh issue create	Create a new issue
Issue	gh issue list	List open issues
Issue	gh issue close 7	Close issue #7
Release	gh release create v1.0	Create a release from a tag
Actions	gh run list	List recent workflow runs
Actions	gh run watch	Watch a live workflow run

■ **'gh pr create'** automatically detects your branch, suggests a title from commits, and lets you pick reviewers.

13. .gitignore Patterns

Advanced ignore rules

.gitignore tells Git which files to never track. Your existing notes cover the basics. Here are the advanced patterns you need to know:

```
● ● ● # Advanced .gitignore patterns
# Ignore a specific file:
secret.env

# Ignore all .env files anywhere in project:
**/.env

# Ignore a folder:
node_modules/

# Ignore all .log files:
*.log

# BUT track one specific log file:
!important.log

# Ignore everything in a folder except .gitkeep:
uploads/*
!uploads/.gitkeep

# Ignore files only at root level (not subdirs):
/TODO.md

# Ignore a folder at any depth:
**/temp/
```

```
● ● ● # Common fixes
# Fix: already-tracked file not being ignored:
git rm --cached secret.env
echo "secret.env" >> .gitignore
git commit -m "Stop tracking secret.env"

# Create .gitignore for your language/framework:
# Visit: https://gitignore.io
# or use: gh repo create --gitignore Python
```

■ ■ Adding a file to .gitignore does NOT stop tracking it if it was already committed. Use 'git rm --cached'.

14. GitHub Platform Features

Issues · Actions · Pages · Releases

Issues

Track bugs, features, and tasks. Can reference commits and PRs. Use labels, milestones, and assignees. Close issues automatically via commit: "Fixes #42" in commit message.

Pull Requests

The core collaboration unit. Request code review, discuss changes inline, link to issues, require checks to pass before merge. Use draft PRs for work in progress.

GitHub Actions

CI/CD built into GitHub. Workflows in `.github/workflows/*.yml` trigger on push, PR, schedule. Auto-run tests, build Docker images, deploy to cloud, all for free on public repos.

GitHub Pages

Host static websites directly from a repo. Push HTML/CSS/JS to `gh-pages` branch or `docs/` folder. Your site lives at `username.github.io/repo-name`. Supports Jekyll natively.

Releases

Package your software with a tag, release notes, and binary attachments. Tied to annotated tags. GitHub notifies watchers and shows the latest release on the repo homepage.

Branch Protection

Set rules on main: require PR reviews, require status checks, prevent force pushes, require linear history. Found under Settings → Branches → Add rule.

```
● ● ● # GitHub features via CLI
# Enable GitHub Pages via CLI:
gh repo edit --enable-pages --pages-branch gh-pages

# Create a release:
git tag -a v1.0 -m "First release"
git push origin v1.0
gh release create v1.0 --notes "What changed in this version"
```

. Quick Reference Card

The commands from this PDF at a glance

Command	What it Does
<code>git reflog</code>	See full history of HEAD movements
<code>git cherry-pick <hash></code>	Copy one commit to current branch
<code>git revert <hash></code>	Safe undo — adds inverse commit
<code>git commit --amend --no-edit</code>	Add forgotten file to last commit
<code>git tag -a v1.0 -m "msg"</code>	Create annotated release tag
<code>git push origin --tags</code>	Push all tags to remote
<code>git bisect start/good/bad</code>	Binary search for buggy commit
<code>git blame -L 20,40 file.js</code>	Who wrote lines 20-40 of file.js
<code>git log --oneline --graph --all</code>	Visual branch graph
<code>git clean -fd</code>	Delete all untracked files+dirs
<code>git config --global alias.lg ...</code>	Create command shortcuts
<code>ssh-keygen -t ed25519 -C email</code>	Generate SSH key for GitHub
<code>gh pr create</code>	Create PR from CLI
<code>gh issue list</code>	List open issues from CLI
<code>git rm --cached file</code>	Untrack already-tracked file